COMP-4002 Assessment 3 Report

Name: Café Sales Analytics & Revenue Prediction
Group: 1
Team:

- Andrea Bozzelli
- Ticiana Galajxhi
- Niccolò Tomassini
- Natalia Iakovleva
- Benjamin Muscat

Introduction

This report documents the end-to-end machine learning pipeline developed as the final project for COMP-4002 (Introduction to Machine Learning). The goal is to apply core ML concepts like data collection, preprocessing, exploratory data analysis, model selection, training and evaluation to a real-world business problem.

The dataset used is a café sales record containing 10,000 transactions from a single café across the full year of 2023. The raw data is intentionally dirty, featuring missing values, sentinel strings (`"ERROR"` , `"UNKNOWN"`), and inconsistent formatting, making it a realistic starting point for a full data science workflow.

The project is structured into four stages:

1. **Data preprocessing** — cleaning, imputation, and feature engineering
2. **Exploratory data analysis (EDA)** — understanding temporal patterns, product mix, and customer behaviour
3. **Model selection and training** — building and comparing classification models to predict daily revenue levels
4. **Evaluation and reflection** — comparing model performance and identifying key drivers of predictive accuracy

The central prediction task asks: *can we classify whether a given day will be a high-revenue or low-revenue day, based on temporal and lag-based features?* This framing transforms the problem into a

binary classification task, making it well-suited to the supervised learning techniques covered in COMP-4002.

Data Preprocessing

Overview

The raw dataset (`dirty_cafe_sales.csv`) contains **10,000 transactions** across 9 columns covering item, quantity, price, payment method, location, date, and time. It is intentionally noisy: missing values are encoded as empty strings, `"ERROR"` , `"UNKNOWN"` , or `"NULL"` rather than standard `NaN` , and numeric columns contain non-numeric sentinel strings. The full preprocessing pipeline is implemented in `data_preprocessing.ipynb` .

Key Issues & Missing Data

After normalising all sentinel values to `NaN` , the missing-value profile is:

Column	Missing %
Location	39.6 %
Payment Method	31.8 %
Item	9.7 %
Price Per Unit	5.3 %
Transaction Time	5.1 %
Total Spent	5.0 %
Quantity	4.8 %
Transaction Date	4.6 %

Missingness is scattered randomly across rows with no systematic pattern, which validates a column-by-column imputation approach.

Imputation Strategy

Each column received a tailored strategy based on its semantics:

- **Item** — mode imputation (8 discrete menu items; most frequent is a reasonable default)
- **Quantity** — median imputation (integer values 1–5; robust to mild skew)
- **Price Per Unit** — mean per item, applied in two passes: first where the item is already known, then again after item imputation fills remaining gaps
- **Total Spent** — recalculated as `Quantity × Price Per Unit` after each imputation pass, ensuring internal consistency rather than imputing a raw number
- **Payment Method / Location** — mode imputation (no business rule links them to other columns)
- **Transaction Date** — rows dropped (~4.6 %); dates cannot be reliably reconstructed
- **Transaction Time** — median hour imputation to preserve the realistic time distribution

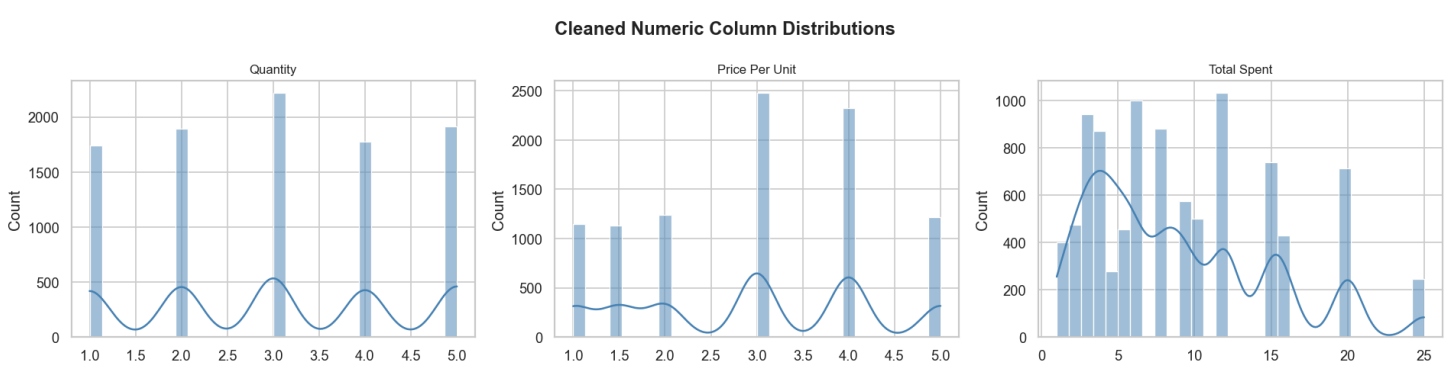
The two-pass approach for `Price Per Unit` is necessary because some rows are missing both `Item` and `Price Per Unit` simultaneously; filling item first unlocks price imputation for those rows.

After the full pipeline, **zero missing values** remain.

Cleaning Steps

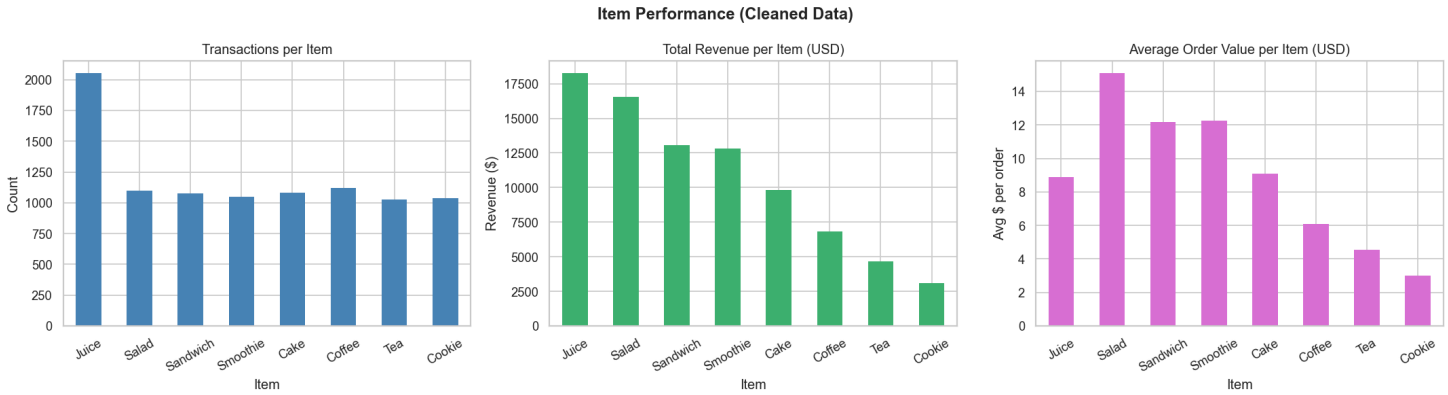
- **Type casting**: numeric columns cast via `pd.to_numeric(errors="coerce")` ; dates via `pd.to_datetime(errors="coerce")` ; categorical columns converted to the `category` dtype
- **Duplicate removal**: no full-row duplicates or duplicate `Transaction ID` s were found (9,540 rows retained)
- **Outlier capping**: IQR-based capping applied to `Quantity` , `Price Per Unit` , and `Total Spent` ; `Quantity` and `Price Per Unit` had no significant outliers (bounded 1–5 by design); a small number of `Total Spent` values above £25 were capped

Cleaned Numeric Distributions



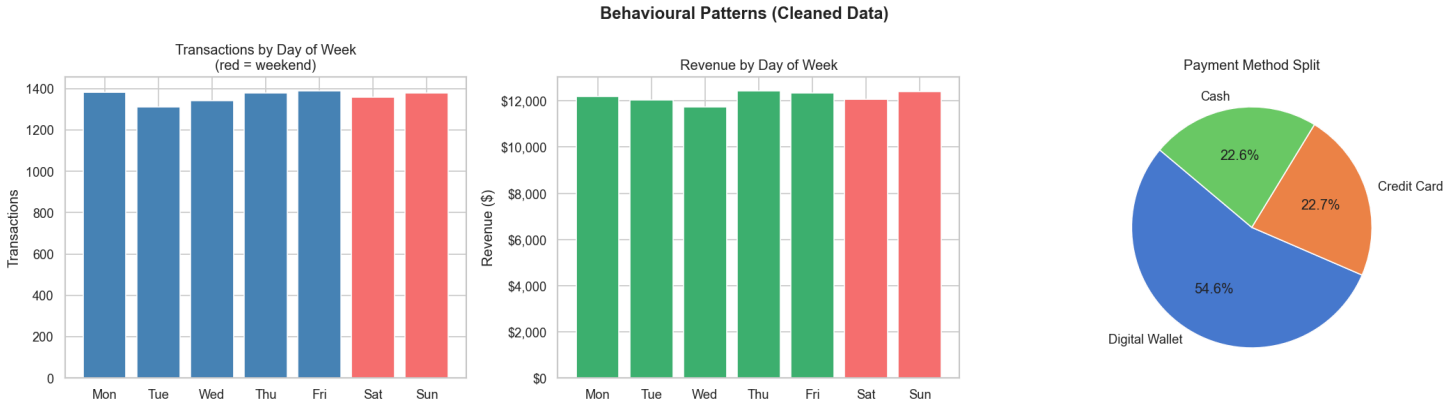
- **Quantity**: roughly uniform across 1–5 (integer peaks)
- **Price Per Unit**: discrete prices $1–$5, with $3 and $4 most frequent
- **Total Spent**: multi-modal distribution reflecting the product of quantity and price, ranging from $1 to $25

Item Performance



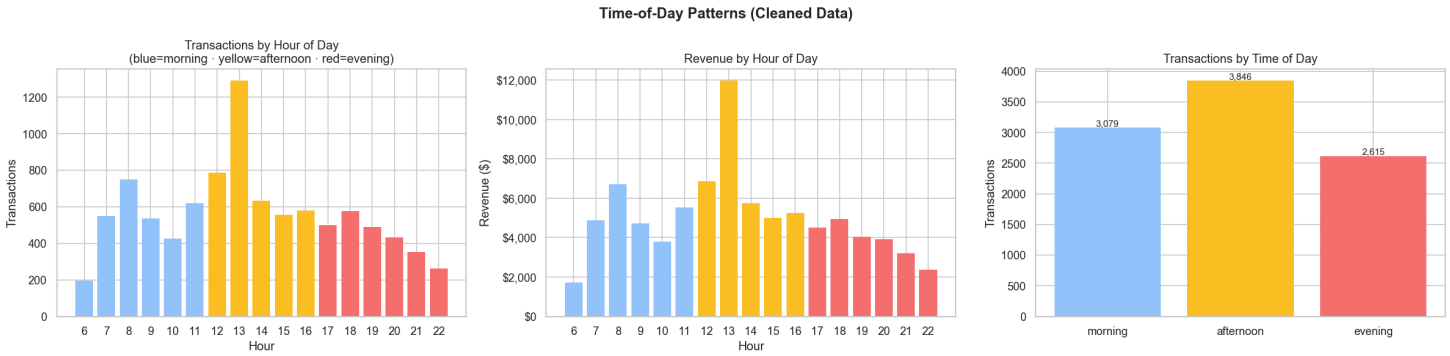
- **Juice** leads in both transaction count (~2,000) and total revenue (~$18,000), partly boosted by mode imputation
- **Salad** and **Sandwich** are the next top performers by revenue, driven by higher average order values (~$12–$15)
- **Cookie** and **Tea** generate the least revenue due to lower unit prices

Behavioural Patterns



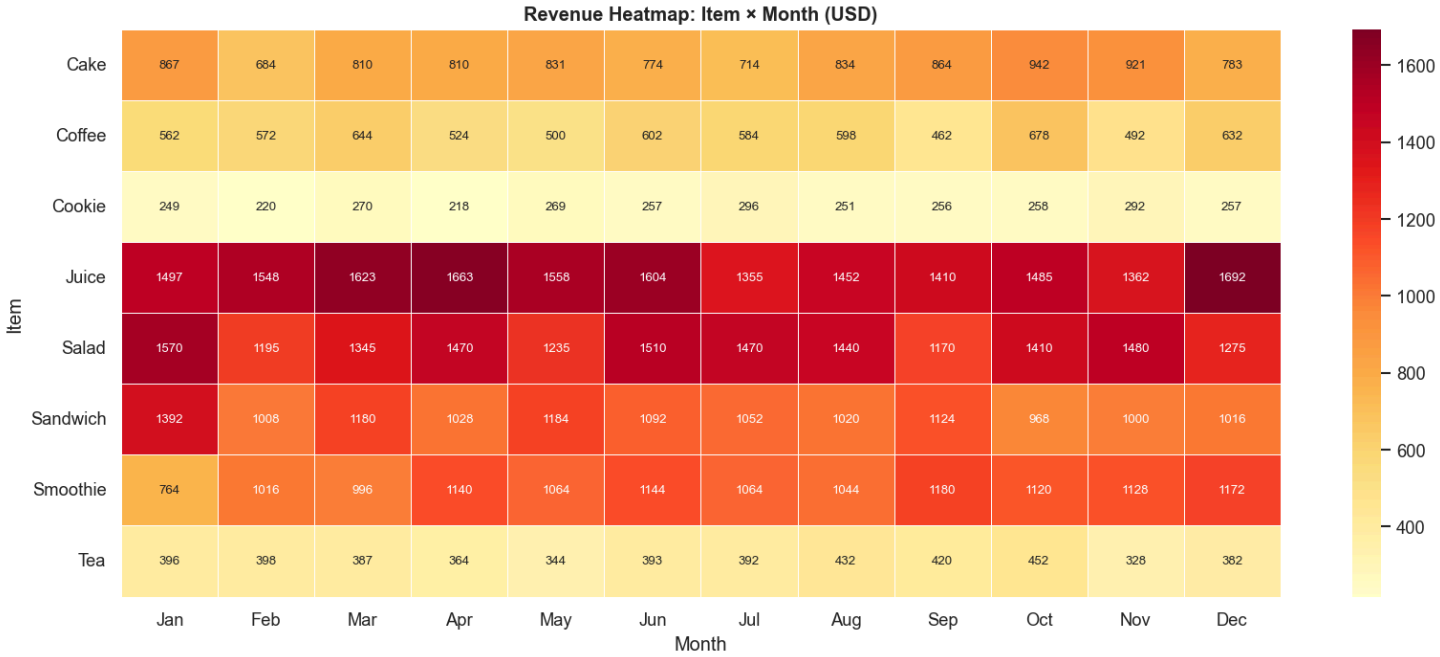
- Transaction volume is **evenly distributed across days of the week** — no strong weekday vs. weekend effect
- **Digital Wallet** dominates payment methods at 54.6 %, followed by Credit Card (22.7 %) and Cash (22.6 %). The high Digital Wallet share is partly due to mode imputation filling ~32 % of missing payment values

Time-of-Day Patterns



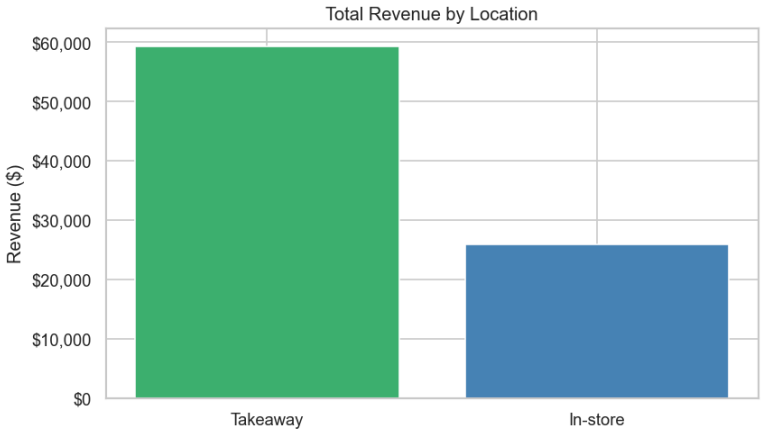
- The **lunch rush (12:00–14:00)** is the clear peak, with hour 13 reaching ~1,300 transactions and ~$12,000 in revenue
- **Morning** (6–11) accounts for ~3,079 transactions, **afternoon** (12–16) for ~3,846, and **evening** (17–22) for ~2,615
- Revenue per hour follows the same pattern, confirming the lunch period as the highest-value time slot

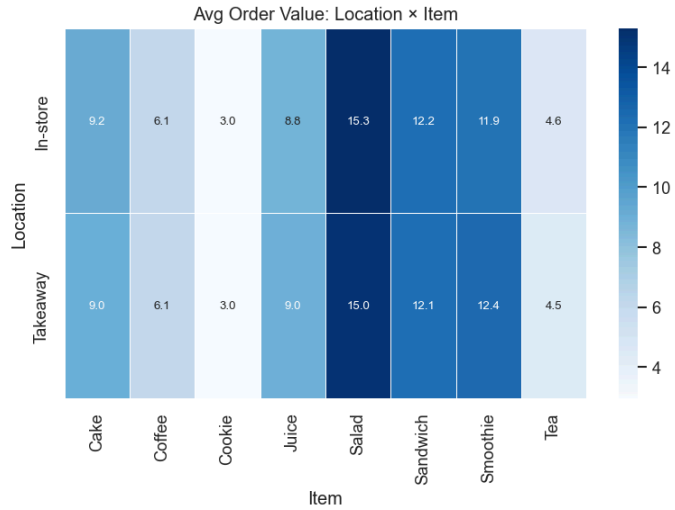
Revenue Heatmap: Item × Month



- **Juice** and **Salad** consistently dominate revenue across all 12 months
- Revenue is relatively stable month-to-month — no strong seasonal effect is visible
- **Cookie** and **Tea** remain the lowest earners throughout the year

Location Breakdown





- **Takeaway** accounts for roughly **70 %** of total revenue (~$59K vs. ~$25K for In-store). The high Takeaway share is partially a result of mode imputation on the Location column
- Average order values are consistent across locations for the same item — the price structure does not vary by location

Feature Engineering

12 new columns were derived to support modelling:

Category	Features
Date-derived	`year` , `month` , `day_of_week` , `is_weekend` , `quarter`
Time-derived	`hour` , `time_of_day` (morning / afternoon / evening)
Payment encoding	`is_cash` , `is_credit_card` , `is_digital_wallet`
Location encoding	`is_takeaway`

These one-hot encoded flags and temporal features make the dataset immediately usable by ML models requiring numeric input.

Final Dataset

The cleaned dataset (`cafe_sales_cleaned.csv`) contains **9,540 rows** and **21 columns** (9 original + 12 engineered), with no missing values. It is ready for EDA and modelling.

Notebook cafemodel.ipynb

Full Documentation docs/data_preprocessing.md

Model Selection and Training

Introduction and problem statement for modelling

The prediction models will focus on answering this question: "Can we predict whether a given day will be a high-revenue or low-revenue day, based on temporal features?" The aim of the prediction would be to understand which time-based factors (such as day of the week, month, seasonality, and holidays) are most strongly associated with revenue fluctuations, and to use these insights to improve planning, forecasting, and decision-making for the cafe business owner.

Dataset description

The dataset that has been used for modelin, contains 9,540 café transactions recorded throughout the full year of 2023 (January 1 to December 31), spread across 20 columns with no missing values. It captures 8 menu items (Juice, Coffee, Salad, Cake, Sandwich, Smoothie, Cookie, and Tea) with Juice being the most frequently purchased.

Each transaction records the quantity (1–5 units), price per unit (£1–£5), and total spent (£1–£25), with an average transaction value of £8.93 and an average daily revenue of £233.

Orders are split between two location types: Takeaway (70%) and In-store (30%). Payment is made via Digital Wallet (55%), Credit Card (23%), or Cash (23%). Transactions are distributed across all times of day, afternoon (40%), morning (32%), and evening (27%) and cover both weekdays (71%) and weekends (29%).

Beyond the raw transactional fields, the dataset includes several pre-engineered features such as hour, month, quarter, day_of_week, time_of_day, and binary flags (is_weekend, is_cash, is_credit_card, is_digital_wallet, is_takeaway).

First model: Logistic regression

To address the problem of predicting whether a given day is a high- or low-revenue day, a Logistic Regression model was selected as the baseline classification approach.

The model estimates the probability that a given day belongs to the high revenue class based on temporal and lag-based features derived from historical data.

We chose to start with this model beacuse:

- Logistic Regression allows clear interpretation of feature effects, making it suitable for understanding how temporal factors can influence revenue outcomes.
- Works as a strong baseline model against which more complex models can later be compared.
- The model produces probability estimates, enabling analysis of prediction confidence and decision thresholds.

Training and validation

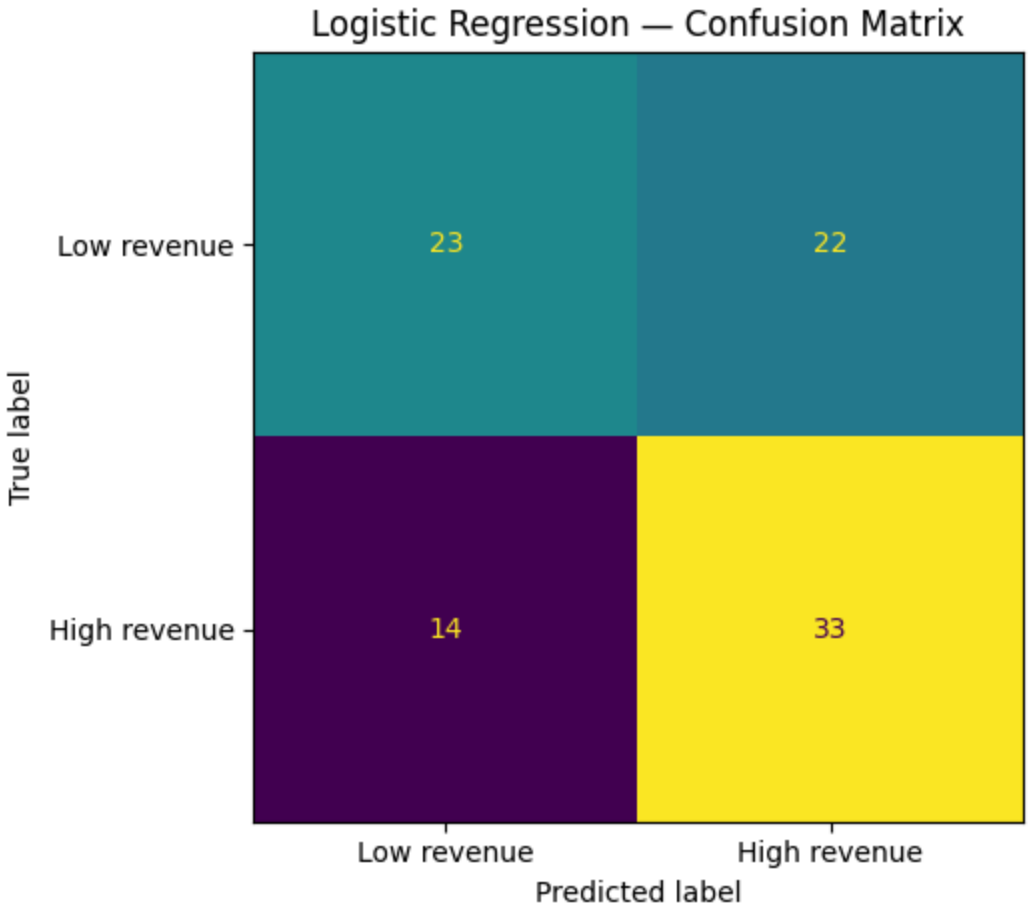
A time-based train-test split was applied:

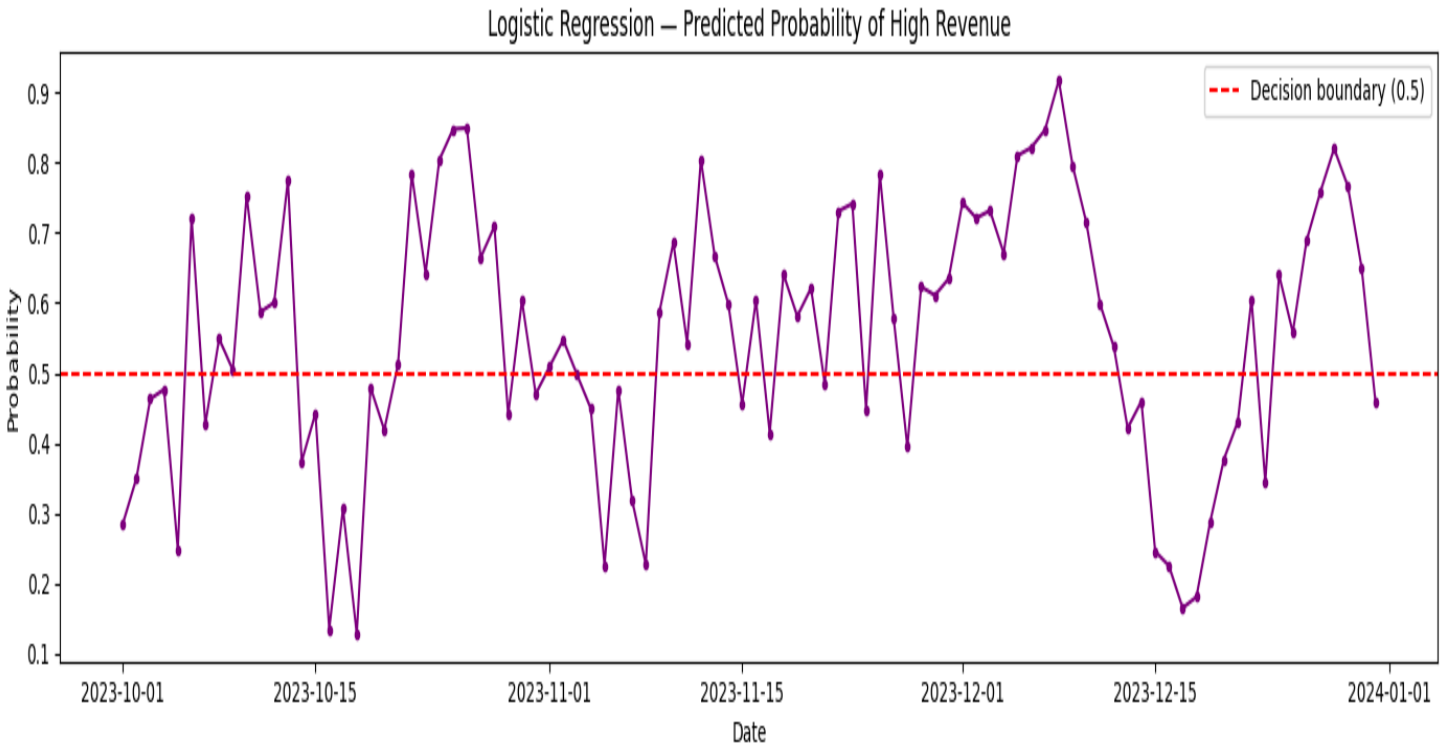
- Training set: January–September 2023
- Test set: October–December 2023

This approach preserves temporal ordering and avoids data leakage, ensuring that the model is evaluated on future, unseen data. However, it would be interesting to see how it performs against other test sets in cross validation folds. Additionally, all input features were standardized using StandardScaler to ensure consistent feature scaling and improve model convergence.

```
Median daily revenue: £230.75
Class distribution:
revenue_class
1    179
0    179
Name: count, dtype: int64

Accuracy: 0.609

Classification Report:
              precision    recall  f1-score   support

 Low revenue       0.62      0.51      0.56        45
High revenue       0.60      0.70      0.65        47

    accuracy                           0.61        92
   macro avg       0.61      0.61      0.60        92
weighted avg       0.61      0.61      0.60        92
```

As shown in the graphs below, there's no meaningful pattern. The model does not reflect "busy periods" vs "quiet periods." Here, the line zigzags unpredictably, which is why accuracy stalled at 64%.


Logistic Regression — Confusion Matrix
Low revenue
High revenue
True label
23
22
14
33
Low revenue
High revenue
Predicted label


Logistic Regression — Predicted Probability of High Revenue
Decision boundary (0.5)
Probability
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
2023-10-01
2023-10-15
2023-11-01
2023-11-15
2023-12-01
2023-12-15
2024-01-01
Date

Logistic regression with more features

Training and validation

Compared to the previous model, we added more lag features to have more signals on expected revenue and help the model understand time patterns. In this case, incorporating lag features, rolling averages, volatility measures, and seasonal indicators allows the model to learn temporal dependencies, trends, and recurring behaviors in daily revenue. The model was evaluated using a single time-based train/test split rather than cross-validation, this approach was chosen because so the model trains on past data and test on future ones.

```
Median daily revenue: £230.50
Class distribution:
revenue_class
0    168
1    167
Name: count, dtype: int64

Accuracy: 0.957

Classification Report:
              precision    recall  f1-score   support

 Low revenue       1.00      0.91      0.95        45
High revenue       0.92      1.00      0.96        47

    accuracy                           0.96        92
   macro avg       0.96      0.96      0.96        92
weighted avg       0.96      0.96      0.96        92
```

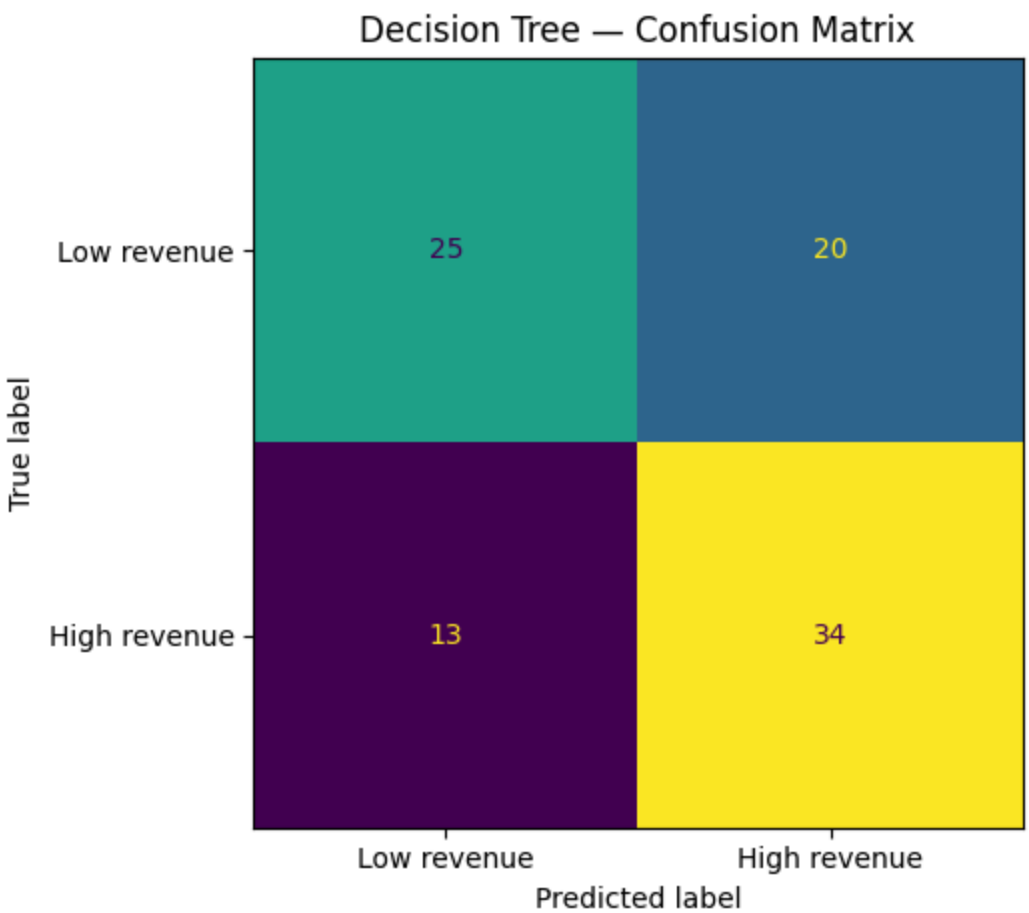
Second model: Decision tree

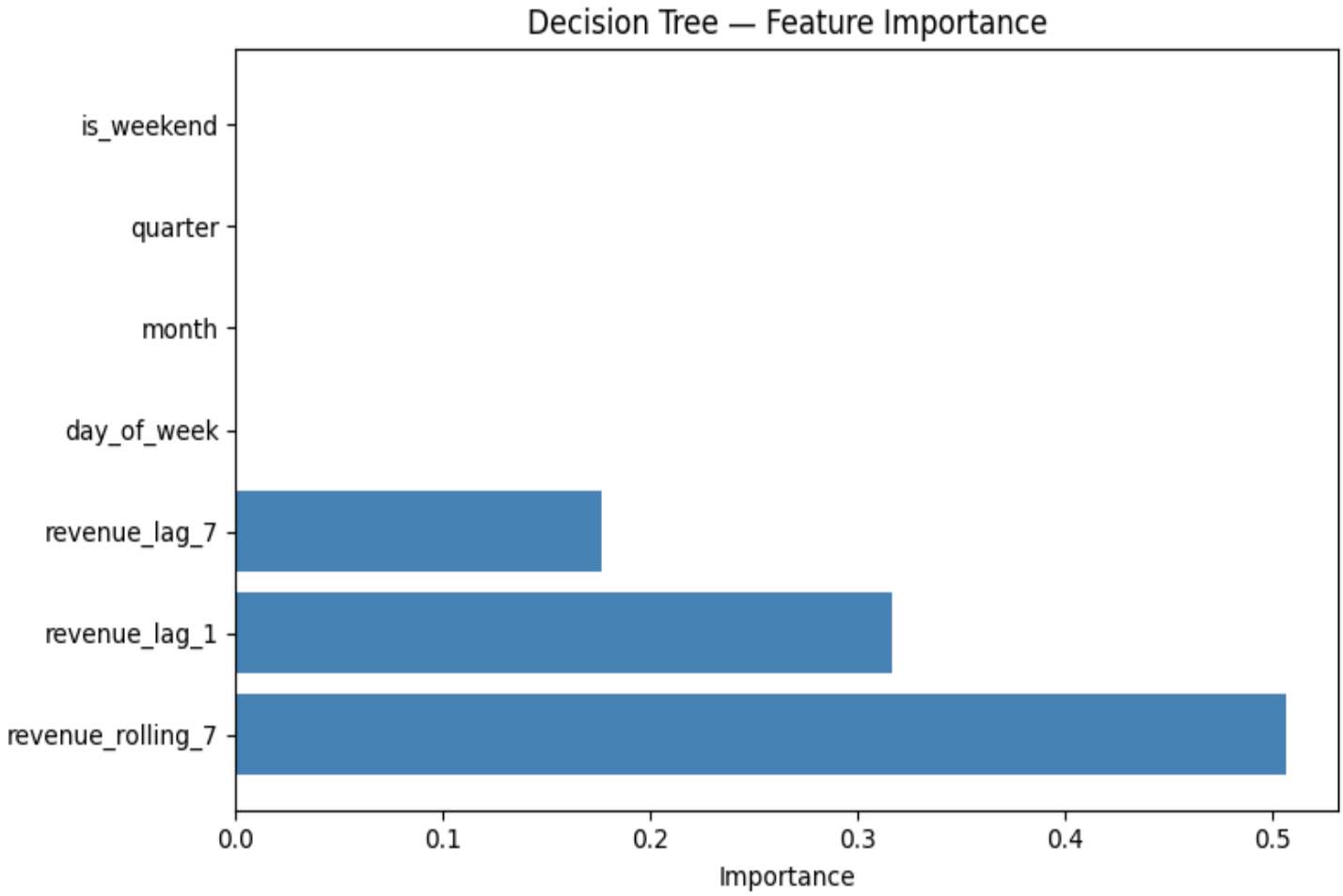
Training and validation

The decision tree model was trained using engineered features including lagged revenue values, rolling averages, and calendar-based variables such as day of the week, month, and weekend indicators. These features were designed to capture both short-term trends and potential seasonal patterns in revenue.

To evaluate the model, a time-based train-test split was used, where the model was trained on earlier observations and validated on later, unseen data. Cross-validation was not used because standard cross-validation techniques randomly shuffle the data, which would break the temporal order and introduce data leakage by allowing future observations to appear in the training set.

```
Train accuracy: 68.8%
Test accuracy:  64.1%

Accuracy: 0.641

Classification Report:
              precision    recall  f1-score   support

 Low revenue       0.66      0.56      0.60        45
High revenue       0.63      0.72      0.67        47

    accuracy                           0.64        92
   macro avg       0.64      0.64      0.64        92
weighted avg       0.64      0.64      0.64        92
```





Hyperparameter optimization

Decision Tree without feature engineering

Training and validation

The decision tree model was trained using the same feature set as the baseline logistic regression model, including basic temporal variables (day of the week, month, quarter, weekend indicator) and simple lag-based features (previous day revenue, 7-day lag, and 7-day rolling average).

These features provide limited short-term memory of past revenue while keeping the model relatively simple and interpretable. Unlike the later experiments, no additional feature engineering such as extended lags, volatility measures, or seasonal flags was applied.

A time-based train-test split was used:

- Training set: January–September 2023

- Test set: October–December 2023

This ensures that the model is trained on past observations and evaluated on future data, preserving temporal order and avoiding data leakage. To improve model performance, hyperparameters were tuned using GridSearchCV. The search explored different values for maximum tree depth, minimum number of samples required to split a node, minimum number of samples in leaf nodes, and the splitting criterion (gini or entropy). The best-performing combination of parameters was selected based on cross-validated accuracy.

Results

The decision tree model achieved a training accuracy of 68.8% and a test accuracy of 64.1%, indicating moderate predictive performance. The relatively small difference between training and test accuracy suggests that the model generalises reasonably well and is not heavily overfitting.

Although hyperparameter tuning was applied, the improvement over the baseline logistic regression remains limited. This suggests that the main constraint is not the model configuration, but the limited predictive power of the available features. Additionally, since GridSearchCV uses standard cross-validation, it may not fully respect the temporal structure of the data, which can lead to slightly optimistic performance estimates.

Overall, the decision tree captures some short-term patterns but does not significantly outperform simpler models when only basic temporal and lag features are used.

The code, along with the corresponding evaluation metrics and visualisations, is available in `./evaluation/cafemodel2.ipynb` in section `Default features/Decision Tree`.

To further validate the hypothesis that model performance is driven primarily by the choice of features rather than the specific model type or its hyperparameters, we repeat the same analysis using a Random Forest model with hyperparameter optimisation, as described below.

Random Forest without feature engineering

Training and validation

The Random Forest model was trained using the same feature set as the baseline models, including basic temporal variables (day of the week, month, quarter, weekend indicator) and simple lag-based features (previous day revenue, 7-day lag, and 7-day rolling average).

Random Forest is an ensemble method that builds multiple decision trees on different subsets of the data and combines their predictions. This typically improves robustness and reduces the risk of overfitting compared to a single decision tree.

A time-based train-test split was applied:

- Training set: January–September 2023
- Test set: October–December 2023

This ensures that the model is trained on past data and evaluated on future observations, preserving the temporal structure of the dataset.

To optimise performance, hyperparameters were tuned using GridSearchCV. The search explored different values for the number of trees, maximum tree depth, minimum samples required for splits and leaves, and the number of features considered at each split. The best-performing configuration was selected based on cross-validated accuracy.

Results

The Random Forest model achieved a training accuracy of 84.6% and a test accuracy of 63.0%. While the training accuracy is notably higher than that of the single decision tree, the test accuracy remains similar, indicating that the ensemble model does not provide a meaningful improvement in generalisation performance.

The classification results show a similar pattern to previous models.

Despite its greater complexity and the use of hyperparameter tuning, the Random Forest model does not significantly outperform simpler models when trained on the same limited feature set. This supports the hypothesis that predictive performance is primarily constrained by the available features rather than the choice of model or its configuration.

Additionally, as with the decision tree, the use of standard cross-validation within GridSearchCV does not fully account for the temporal nature of the data, which may lead to slightly optimistic estimates of model performance.

The code, along with the corresponding evaluation metrics and visualisations, is available in `./evaluation/cafemodel2.ipynb` in section `Default features/Random Forest`.

Overall, the results indicate that increasing model complexity alone is insufficient to achieve better predictions without more informative features.
To further test this hypothesis we perform hyperparmeter optimisations for both Random Forest and

Decision Tree with the additional features that were shown to be important previously. The results of these experiments are summarized in the following sections.

Decision tree with feature engineering

Training and validation

The decision tree model was retrained using an extended set of engineered features to provide richer information about revenue dynamics. In addition to the basic temporal variables (day of the week, month, quarter, weekend indicator), the model includes multiple lag features (1, 7, 14, and 30 days), rolling averages over different time windows, measures of volatility (rolling standard deviations), and indicators of momentum and percentage change.

Additional binary variables were introduced to capture weekly patterns (e.g. specific days of the week) and seasonal effects (e.g. summer months and December). These features allow the model to capture short-term fluctuations, medium-term trends, and recurring seasonal behaviour in the data.

A time-based train-test split was applied:

- Training set: January–September 2023
- Test set: October–December 2023

This preserves temporal ordering and ensures that the model is evaluated on future observations.

Hyperparameters were tuned using GridSearchCV. The search explored different values for tree depth, minimum samples required for splits and leaves, and the splitting criterion. The best-performing configuration was selected based on cross-validated accuracy.

Results

With the addition of engineered features, the decision tree achieved a training accuracy of 98.8% and a test accuracy of 92.4%, representing a substantial improvement over the version without feature engineering. It is also important to note that both classes are predicted with same quality (identical F1-score, differences in precision/recall less than 3%).

However, the very high training accuracy suggests that the model fits the training data extremely closely. Although test performance is also high, these results should be interpreted with caution. Many of the engineered features (such as recent lags, rolling averages, and momentum) are closely related to the target variable in time, which can make the prediction task easier and potentially lead to overly optimistic performance estimates.

In addition, the use of standard cross-validation within GridSearchCV does not fully respect the temporal structure of the data, which may further contribute to optimistic results.

Overall, the experiment demonstrates that feature engineering has a significantly greater impact on model performance than model complexity. By incorporating richer representations of recent revenue behaviour, the model is able to capture patterns that were not detectable using basic temporal features alone. The resulting accuracy is slightly lower than that obtained with the logostic regression on the same feature set. It confirms the idea that the model performance is almost entirely determined by the feature set and not by the model architecture (algorithm type) and its hyperparameters.

Interpretation from an autocorrelation perspective

A key reason for the strong improvement in model performance after feature engineering lies in the autocorrelation structure of the target variable (daily revenue).

Autocorrelation refers to the relationship between a variable and its past values. In time series data such as daily revenue, it is common for observations to be highly dependent on recent history. For example, revenue on a given day is often influenced by revenue on the previous day, the same day in the previous week, or recent trends over several days.

In this dataset, daily revenue likely exhibits strong short-term autocorrelation. This means that values such as revenue_lag_1 (yesterday's revenue), revenue_lag_7 (same day last week), and rolling averages contain information that is highly predictive of the current day's revenue. As a result, when these features are included, the model is effectively given direct signals about the target variable's recent behaviour.

From this perspective, the classification task becomes much easier. The model is no longer relying only on indirect indicators such as day of the week or month, but instead uses variables that are strongly correlated with the target itself. This explains the large increase in accuracy observed after adding engineered features.

However, this also raises an important concern. When features are highly autocorrelated with the target and very close in time, the model may rely heavily on recent values rather than learning more generalisable patterns. Therefore, the observed behaviour suggests that the target variable is strongly autocorrelated, and that the predictive power of the models is driven primarily by this temporal dependence rather than by broader seasonal or calendar-based effects.

The code, along with the corresponding evaluation metrics and visualisations, is available in `./evaluation/cafemodel2.ipynb` in section `Feature engineering/Decision Tree`.

Finally, we perform the same analysis for Random Forest model.

Random forest with feature engineering

Training and validation

The Random Forest model was retrained using the extended set of engineered features. In addition to the basic temporal variables (day of the week, month, quarter, weekend indicator), the model incorporates multiple lag features (1, 7, 14, and 30 days), rolling averages over different time windows, volatility measures (rolling standard deviations), and indicators of momentum and percentage change.

Additional binary variables were included to capture weekly patterns (specific days of the week) and seasonal effects (e.g. summer months and December). These features provide the model with richer information about short-term fluctuations, medium-term trends, and recurring seasonal behaviour.

A time-based train-test split was applied:

- Training set: January–September 2023
- Test set: October–December 2023

This ensures that the model is trained on past observations and evaluated on future data, preserving the temporal structure of the dataset.

To optimise performance, hyperparameters were tuned using GridSearchCV. The search explored different values for the number of trees, maximum tree depth, minimum samples required for splits and leaves, and the number of features considered at each split. The best-performing configuration was selected based on cross-validated accuracy.

Results

With the inclusion of engineered features, the Random Forest model achieved a training accuracy of 100.0% and a test accuracy of 93.5%, making it the best-performing model among all those tested apart from logistic regression described before (the difference is ~2%, which might not be statistically significant and is definitely not important from practical perspective).
The classification results show strong and balanced performance across both classes.

However, the perfect training accuracy suggests that the model fits the training data extremely closely. Although the test performance is also high, these results should be interpreted with caution.

Some of the engineered features are closely related to the target variable in time, which may make the prediction task easier and lead to overly optimistic performance estimates.

In addition, the use of standard cross-validation within GridSearchCV does not fully preserve the temporal ordering of the data, which may further contribute to optimistic evaluation results.

Overall, the experiment reinforces that feature engineering has a significantly greater impact on model performance than model choice alone. The Random Forest benefits from the richer feature set, but the performance gains are primarily driven by the additional information provided by the engineered variables rather than the complexity of the model itself.

The code, along with the corresponding evaluation metrics and visualisations, is available in `./evaluation/cafemodel2.ipynb` in section `Feature engineering/Random forest`

Evaluation

Overall, a key challenge across all models is that revenue remains relatively constant over time, making it difficult to distinguish clearly between high- and low-revenue days. Because of this limited variability, the models struggle to identify strong patterns, which reduces their predictive performance. This suggests that the current dataset may not contain enough informative variation, and that additional data or more diverse features are needed to improve the reliability of predictions. Here is a brief summary of the models result.

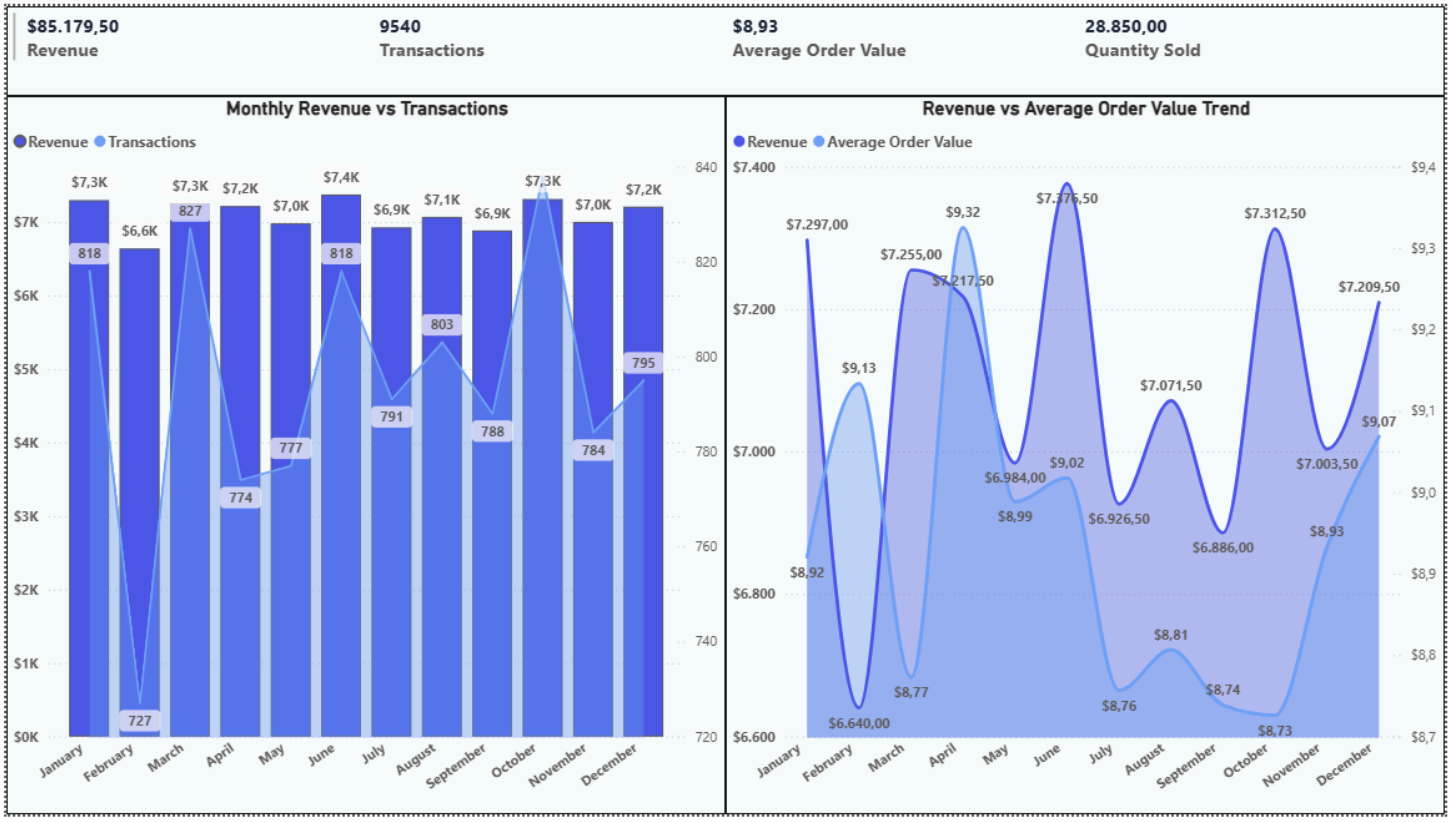
The model for Logistic Regression model achieved 60.9% accuracy and showed a tendency to predict high-revenue days more effectively than low-revenue days. While it identified 70% of high-revenue days, it only captured 51% of low-revenue days, indicating an optimistic bias. This imbalance suggests the model may overestimate demand, which could lead to inefficient business decisions.

The Logistic Regression model with additional features achieved very high accuracy (around 98%) and strong performance across both classes. However, this is likely due to data leakage or overly correlated features, making the results overly optimistic and not reliable for real-world use.

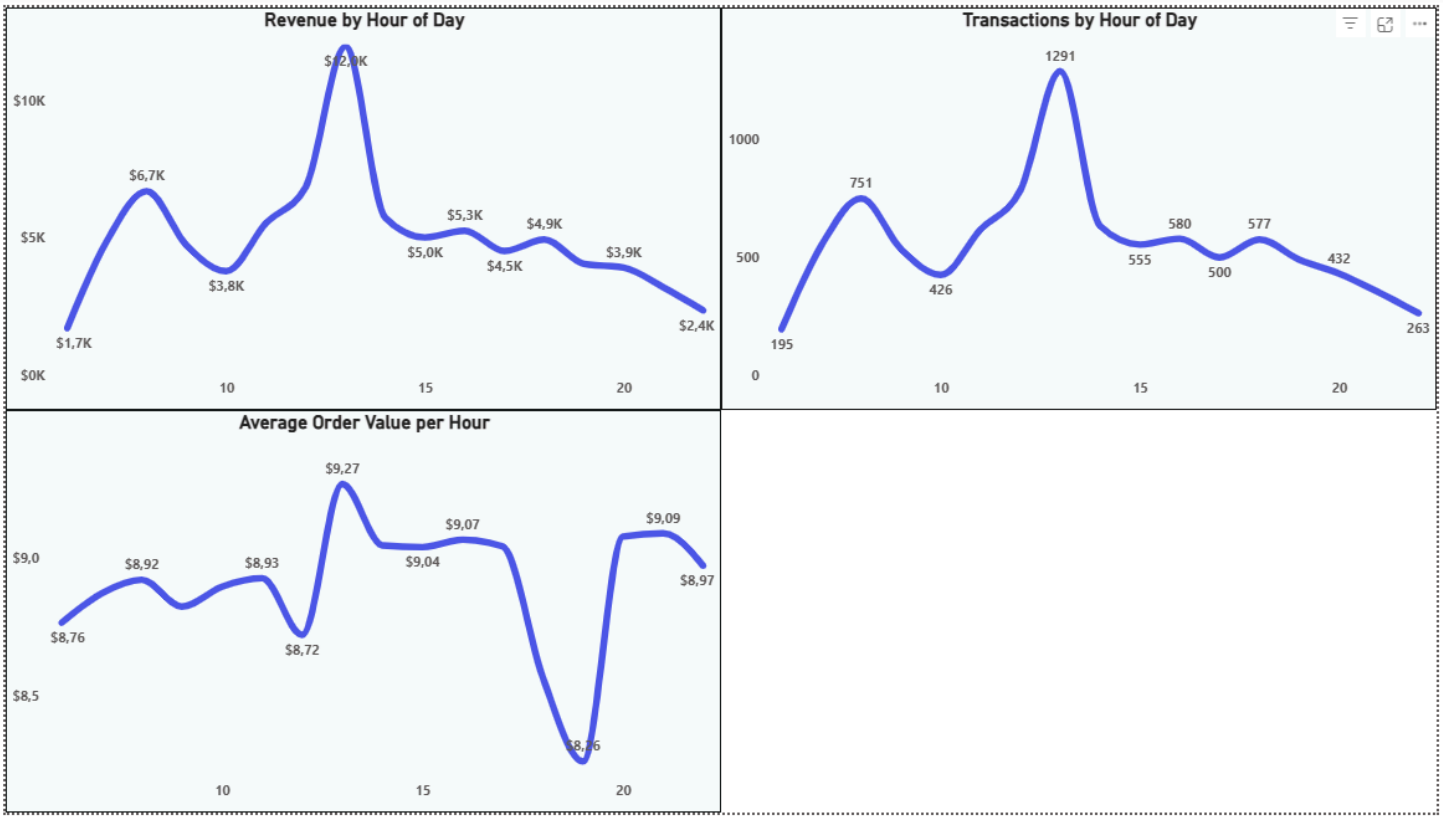
The Decision Tree model performed slightly better than the baseline Logistic Regression, with 64.1% test accuracy and a small gap between training and testing results, indicating good generalization. It relies mainly on recent revenue trends rather than calendar features, suggesting it captures short-term patterns but may miss broader seasonal effects.

Appendix A — EDA Charts

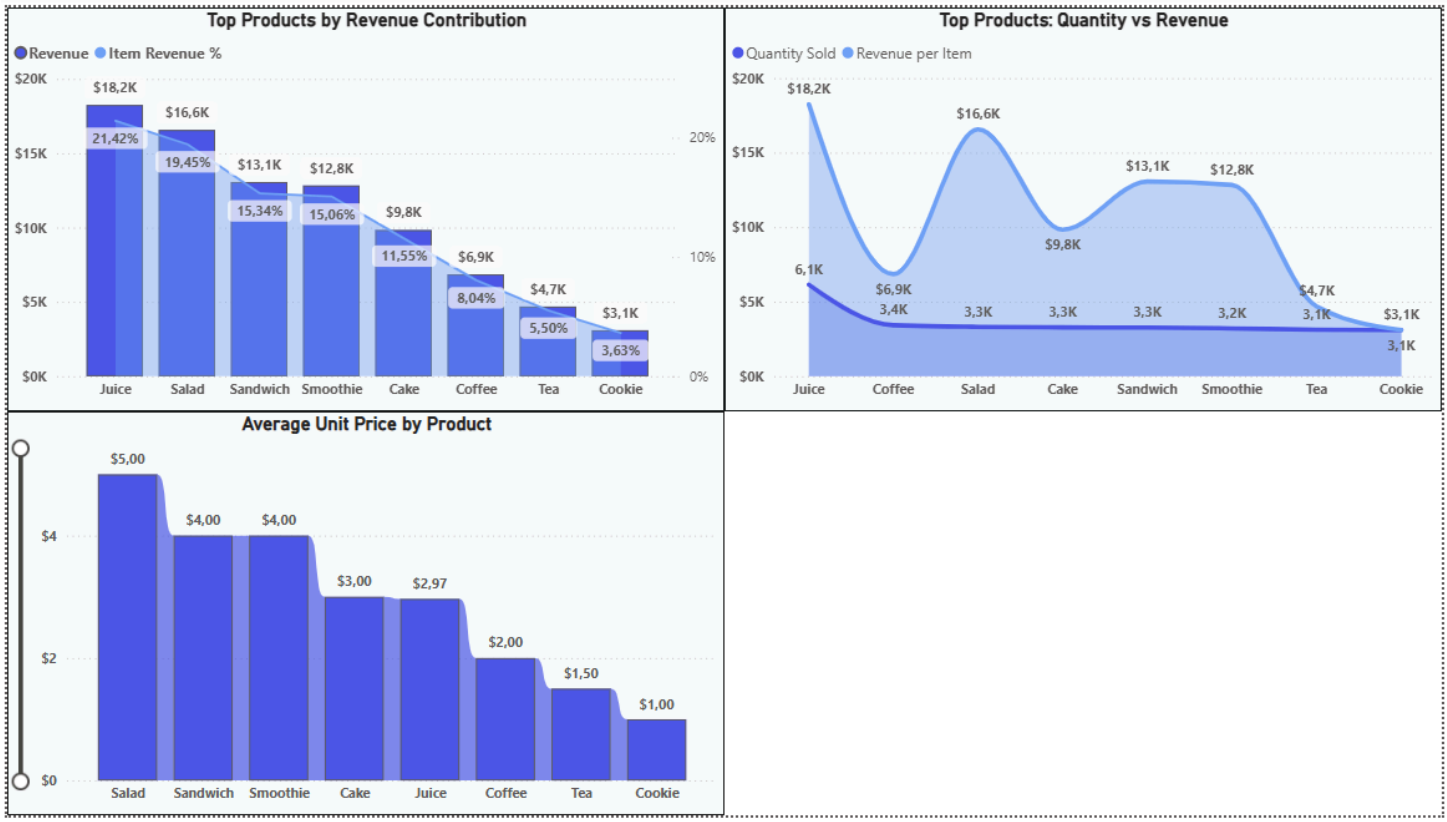
1. Monthly Revenue, Transactions, and Average Order Value Trend



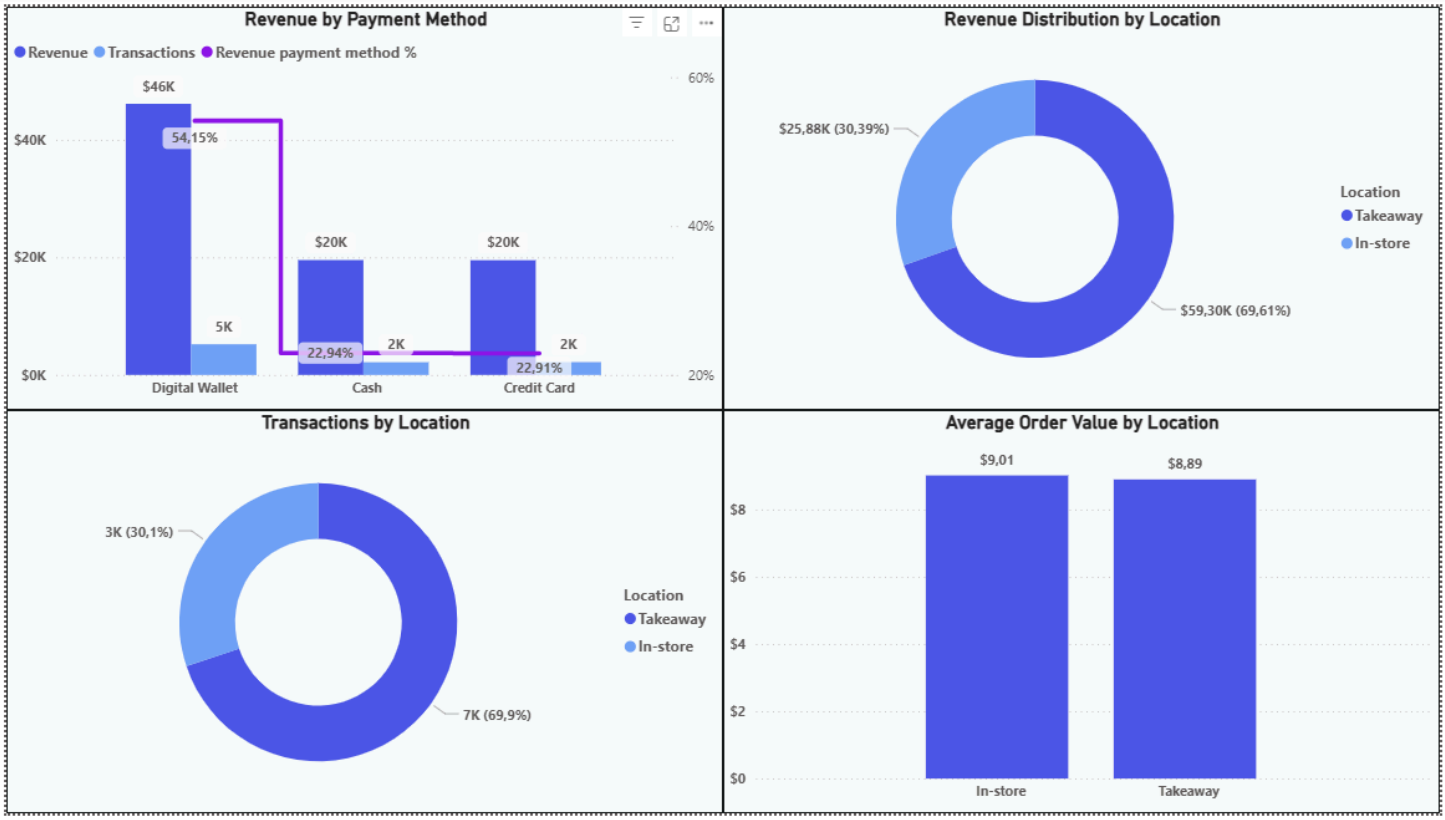
2. Revenue and Transactions by Hour of Day



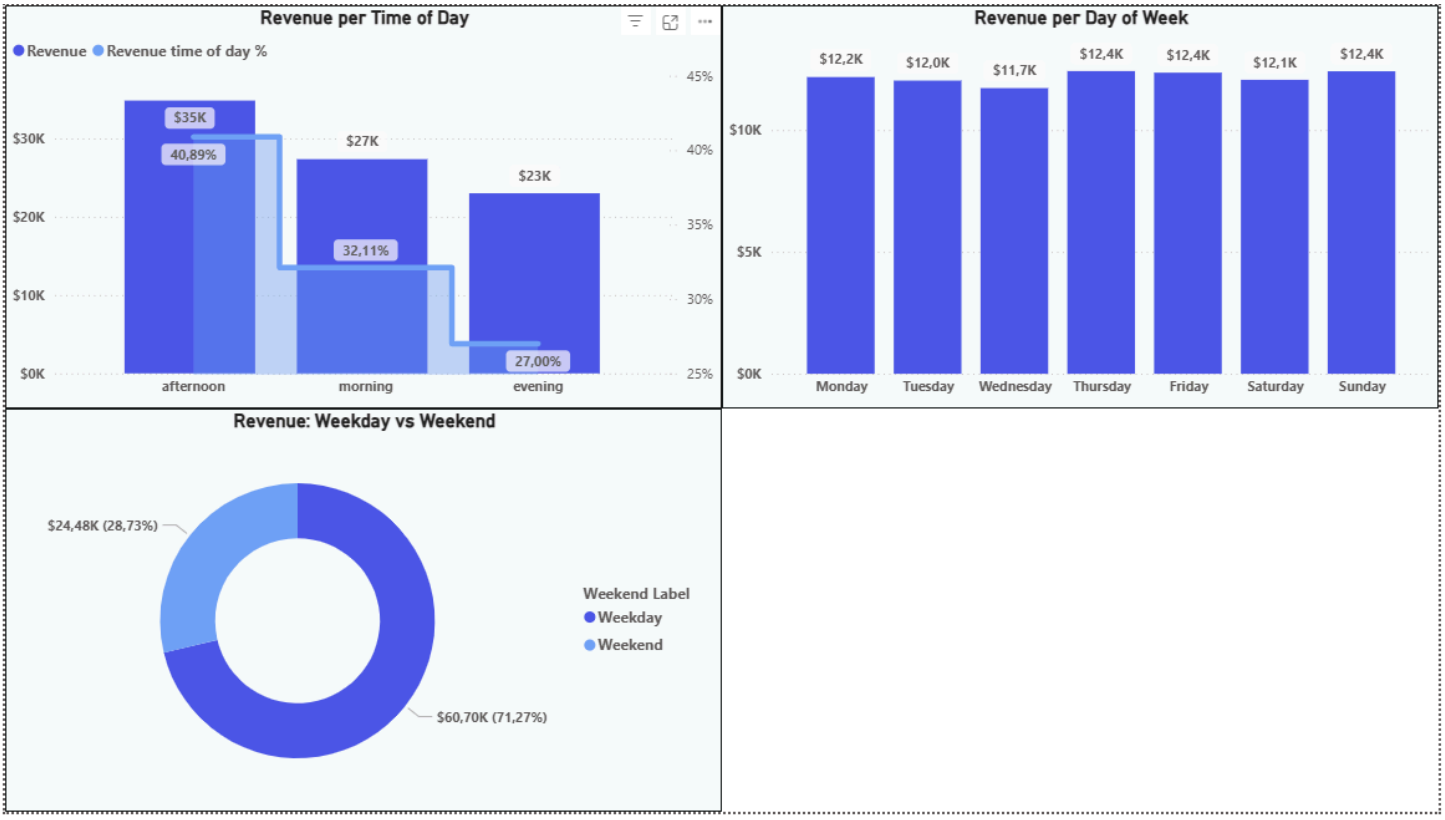
3. Top Products by Revenue Contribution and Average Unit Price



4. Revenue by Payment Method and Location



5. Revenue by Time of Day, Day of Week, and Weekday vs Weekend



6. Revenue Heatmap: Hour vs Day of Week

Revenue Heatmap: Hour vs Day

Day Name	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
Monday	$293,50	$695,50	$999,00	$733,50	$561,00	$750,50	$953,00	$1.727,00	$793,00	$853,50	$757,50	$589,00	$724,00	$449,00	$520,50	$452,50	$323,50
Tuesday	$215,50	$759,50	$863,50	$652,50	$430,00	$808,00	$1.111,50	$1.566,00	$937,00	$703,50	$591,00	$733,00	$583,50	$619,50	$541,00	$522,50	$393,00
Wednesday	$166,00	$577,00	$906,00	$683,00	$687,50	$788,00	$976,50	$1.604,00	$846,50	$548,50	$686,00	$586,00	$700,00	$603,50	$579,50	$496,00	$288,00
Thursday	$359,00	$684,00	$969,00	$701,50	$417,00	$661,00	$953,50	$1.765,00	$867,50	$892,00	$872,50	$532,50	$809,50	$627,50	$541,00	$430,00	$338,50
Friday	$190,00	$660,00	$900,00	$717,00	$727,00	$885,50	$969,00	$1.780,00	$739,50	$657,50	$682,50	$719,50	$767,00	$562,50	$558,00	$471,50	$368,50
Saturday	$249,50	$645,00	$987,50	$631,50	$354,00	$762,50	$936,00	$1.910,50	$695,00	$745,00	$994,00	$798,50	$591,50	$475,50	$581,00	$385,50	$319,50
Sunday	$235,50	$867,00	$1.074,50	$610,00	$613,00	$879,00	$954,50	$1.614,00	$856,50	$617,00	$675,00	$562,50	$766,50	$718,00	$600,50	$441,50	$328,50